Agile Development Process and Devops (ISWE406L).

LAB-FAT

PROJECT – EXPENSE TRACKING SYSTEM

NAME: HEMANATH R

REG.NO: 23MIS0552

CLASS NO: 2398

SET 6

ANSWER ALL THE QUESTIONS
Total Marks: 50

Expense Tracking System

Problem Statement:

Ensure user expenses are recorded and calculated correctly.

Tasks:

1. Develop expense tracking logic

2. Write JUnit tests for calculations

3. Implement CI/CD pipeline with automated testing

- Set up version control using GitHub
- Configure CI pipeline (e.g., Jenkins/GitHub Actions) to trigger on code commits
- Automate build process using Maven

- Integrate automated unit tests in pipeline
- Configure CD pipeline for automatic deployment to staging/production

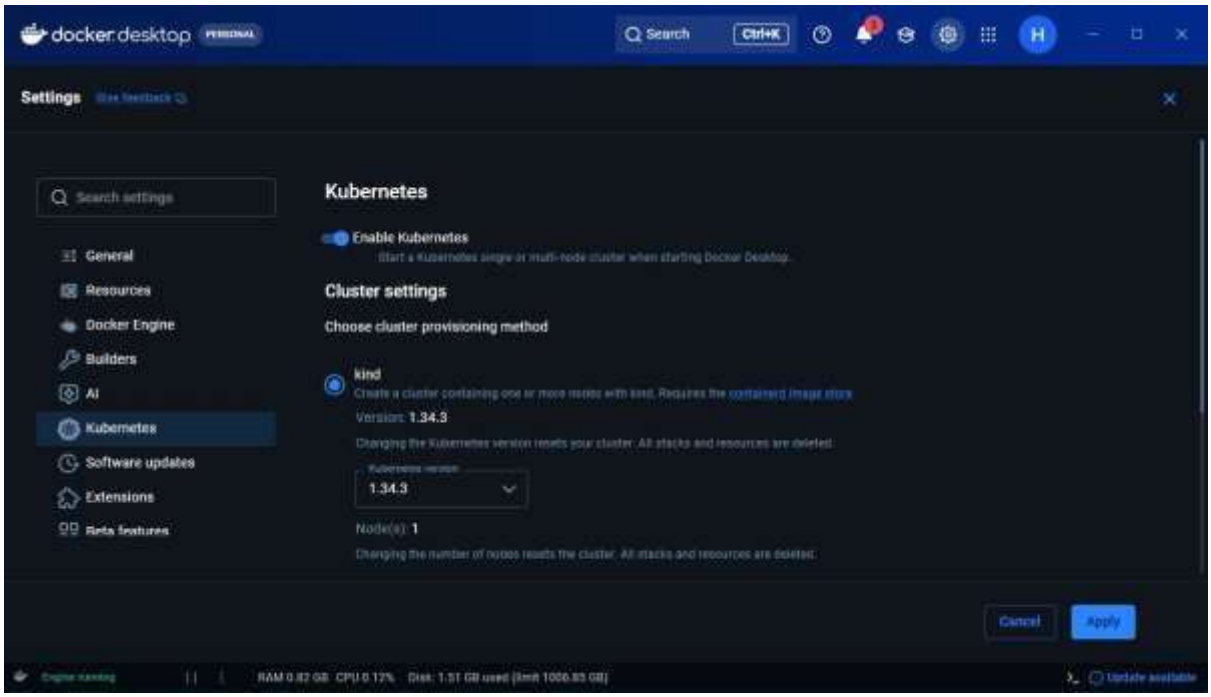
---

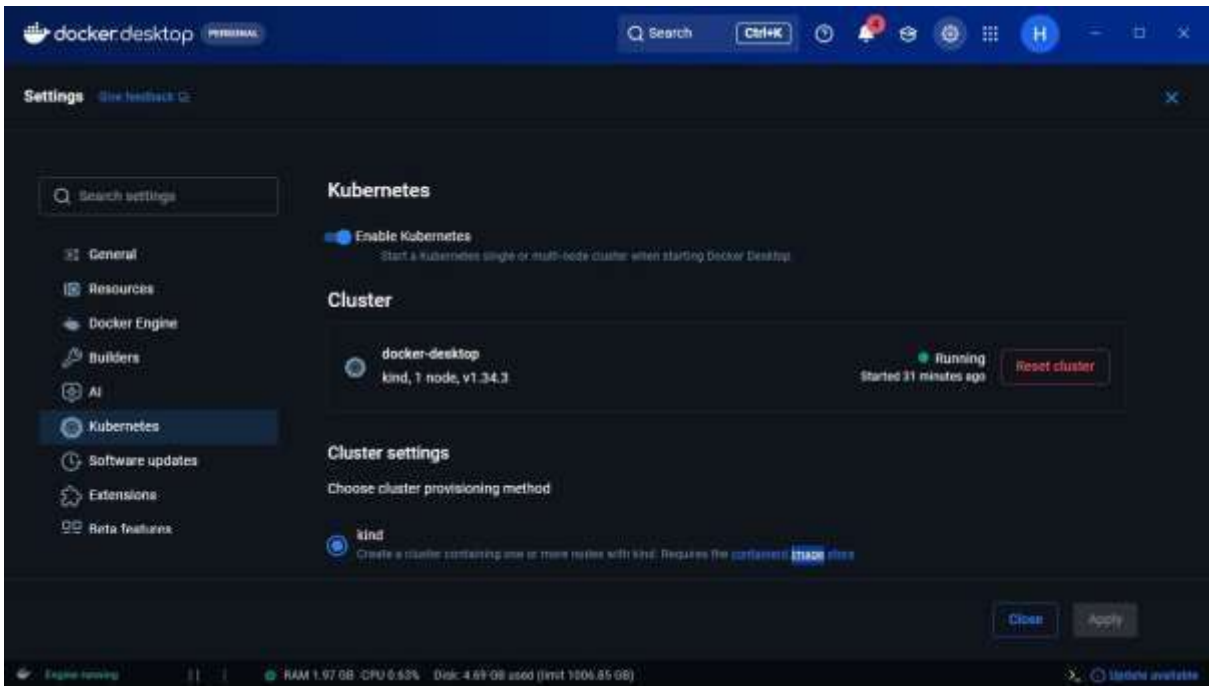
4. Deploy services on Docker and Kubernetes

- Containerize application using Docker (write Dockerfile)
- Build and push Docker image to container registry (Docker Hub)
- Create Kubernetes deployment and service YAML files

1: Install Docker Desktop

Install and enable Kubernetes, enable three plugins (Docker, Docker Pipeline, Kubernetes CLI) and install them





2: Create Java Application (in local machine)

```
expense-tracking-system/
├── src/main/java/com/expense/
|   ├── Expense.java
|   ├── ExpenseTracker.java
|   └── Main.java
├── src/test/java/com/expense/
|   ├── ExpenseTest.java
|   └── ExpenseTrackerTest.java
├── pom.xml (Maven config with JUnit5, JaCoCo)
├── Dockerfile
├── k8s/deployment.yaml
├── .github/workflows/ci-cd-pipeline.yml
├── README.md (Complete documentation)
├── QUICK_START.md (5-minute setup)
├── SETUP_GIT.md (GitHub integration)
└── .gitignore
```

ExpenseTracker.java

```
package com.expense;

import java.time.LocalDate;
import java.time.YearMonth;
import java.util.*;
import java.util.stream.Collectors;

/**
 * ExpenseTracker manages all expense operations and calculations
 */
public class ExpenseTracker {
    private List<Expense> expenses;
    private int expenseCounter;

    public ExpenseTracker() {
        this.expenses = new ArrayList<>();
        this.expenseCounter = 0;
    }

    /**
     * Add a new expense
     */
    public Expense addExpense(String category, double amount, String description, LocalDate date) {
        if (amount < 0) {
            throw new IllegalArgumentException("Amount cannot be negative");
        }
        if (category == null || category.isEmpty()) {
            throw new IllegalArgumentException("Category cannot be empty");
        }

        String expenseId = "EXP" + System.currentTimeMillis() + "_" + (++expenseCounter);
        Expense expense = new Expense(expenseId, category, amount, description, date);
        expenses.add(expense);
```

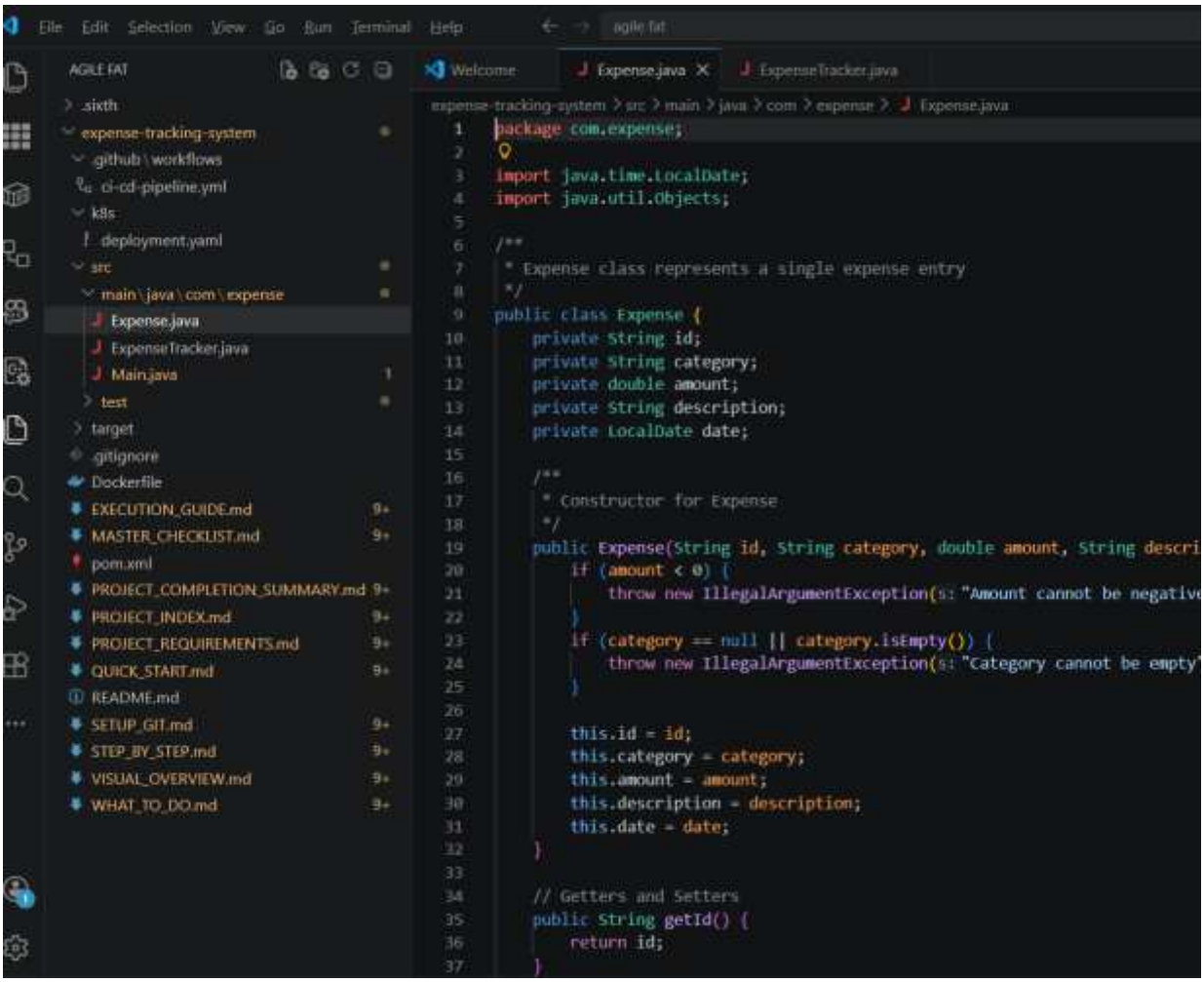
```
    return expense;
}

/**
 * Remove an expense by ID
 */
public boolean removeExpense(String id) {
    return expenses.removeIf(exp -> exp.getId().equals(id));
}

/**
 * Get total expenses
 */
public double getTotalExpenses() {
    return expenses.stream().mapToDouble(Expense::getAmount).sum();
}

/**
 * Get total expenses by category
 */
public double getTotalByCategory(String category) {
    return expenses.stream()
            .filter(exp -> exp.getCategory().equalsIgnoreCase(category))
            .mapToDouble(Expense::getAmount)
            .sum();
}

/**
 * Get expenses by category
 */
public List<Expense> getExpensesByCategory(String category) {
    return expenses.stream()
            .filter(exp -> exp.getCategory().equalsIgnoreCase(category))
            .collect(Collectors.toList());
}
```

```
/**
 * Get expenses within a date range
 */
public List<Expense> getExpensesByDateRange(LocalDate startDate, LocalDate endDate) {
    return expenses.stream()
            .filter(exp -> !exp.getDate().isBefore(startDate) && !exp.getDate().isAfter(endDate))
            .collect(Collectors.toList());
}

/**
 * Get total expenses for a specific month
 */
public double getTotalByMonth(YearMonth month) {
    return expenses.stream()
            .filter(exp -> YearMonth.from(exp.getDate()).equals(month))
            .mapToDouble(Expense::getAmount)
            .sum();
}

/**
 * Get average expense amount
 */
public double getAverageExpense() {
    if (expenses.isEmpty()) {
        return 0;
    }
    return getTotalExpenses() / expenses.size();
}

/**
 * Get category-wise breakdown
 */
public Map<String, Double> getCategoryBreakdown() {
    return expenses.stream()
```

```
        .collect(Collectors.groupingBy(
            Expense::getCategory,
            Collectors.summingDouble(Expense::getAmount)
        ));
}

/**
 * Get all expenses
 */
public List<Expense> getAllExpenses() {
    return new ArrayList<>(expenses);
}

/**
 * Get expense count
 */
public int getExpenseCount() {
    return expenses.size();
}

/**
 * Get highest expense
 */
public Expense getHighestExpense() {
    return expenses.stream()
        .max(Comparator.comparingDouble(Expense::getAmount))
        .orElse(null);
}

/**
 * Get lowest expense
 */
public Expense getLowestExpense() {
    return expenses.stream()
        .min(Comparator.comparingDouble(Expense::getAmount))
```

```
            .orElse(null);
    }

    /**
     * Clear all expenses
     */
    public void clearAllExpenses() {
        expenses.clear();
    }

    /**
     * Update an expense
     */
    public boolean updateExpense(String id, String category, double amount, String description,
LocalDate date) {
        for (Expense exp : expenses) {
            if (exp.getId().equals(id)) {
                exp.setCategory(category);
                exp.setAmount(amount);
                exp.setDescription(description);
                exp.setDate(date);
                return true;
            }
        }
        return false;
    }
}
```

Expense.java



package com.tracking.controller;

import com.tracking.model.Task; import

com.tracking.model.TaskStatus; import

com.tracking.service.TaskService; import

org.springframework.http.HttpStatus; import

org.springframework.http.ResponseEntity; import

org.springframework.web.bind.annotation.*;

import java.util.List;

/**

* REST API controller for task management.

* Provides endpoints for CRUD operations and task tracking.

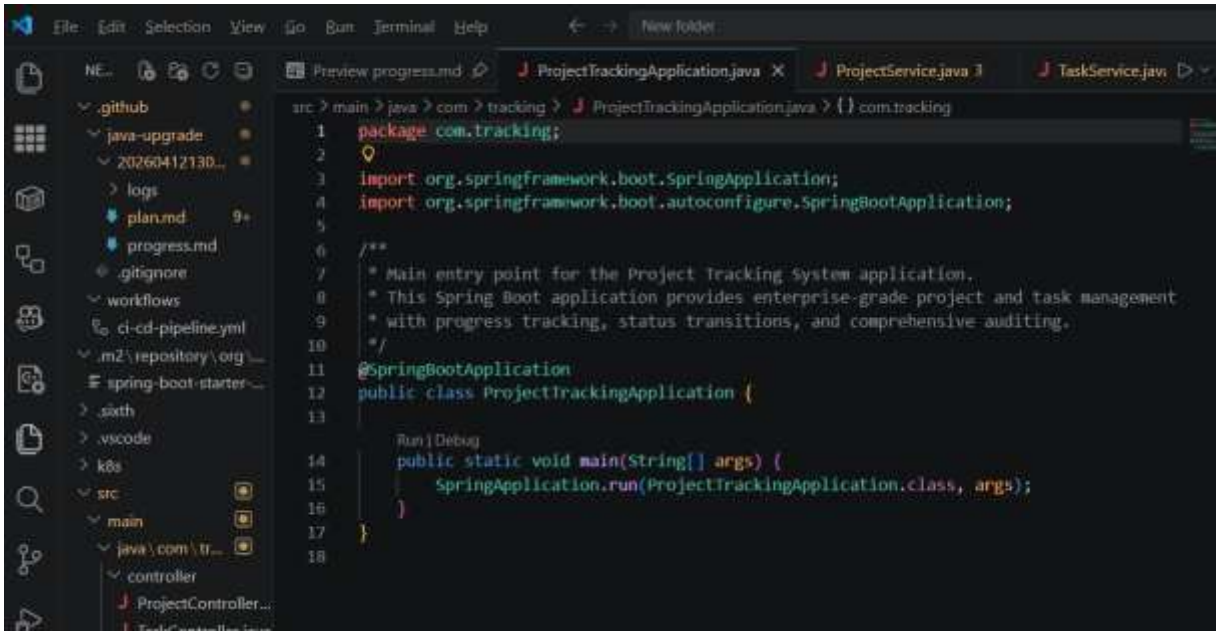
*/

```
@RestController
@RequestMapping("/tasks") public
class TaskController {     private final
TaskService taskService;

   public TaskController(TaskService taskService) {
this.taskService = taskService;
   }

   /**
* Create a new task.
    */
   @PostMapping     public ResponseEntity<Task>
createTask(@RequestBody Task task) {        Task createdTask =
taskService.createTask(task);        return
ResponseEntity.status(HttpStatus.CREATED).body(createdTask);
   }

   /**
* Get a task by ID.
    */
   @GetMapping("/{taskId}")     public ResponseEntity<Task>
getTask(@PathVariable Long taskId) {        Task task =
taskService.getTask(taskId);        return ResponseEntity.ok(task);
   }

   /**
* Get all tasks for a project.
    */
   @GetMapping("/project/{projectId}")     public ResponseEntity<List<Task>>
getTasksByProject(@PathVariable Long projectId) {        List<Task> tasks =
taskService.getTasksByProject(projectId);        return ResponseEntity.ok(tasks);
   }

   /**
* Get all tasks with a specific status.
    */
```

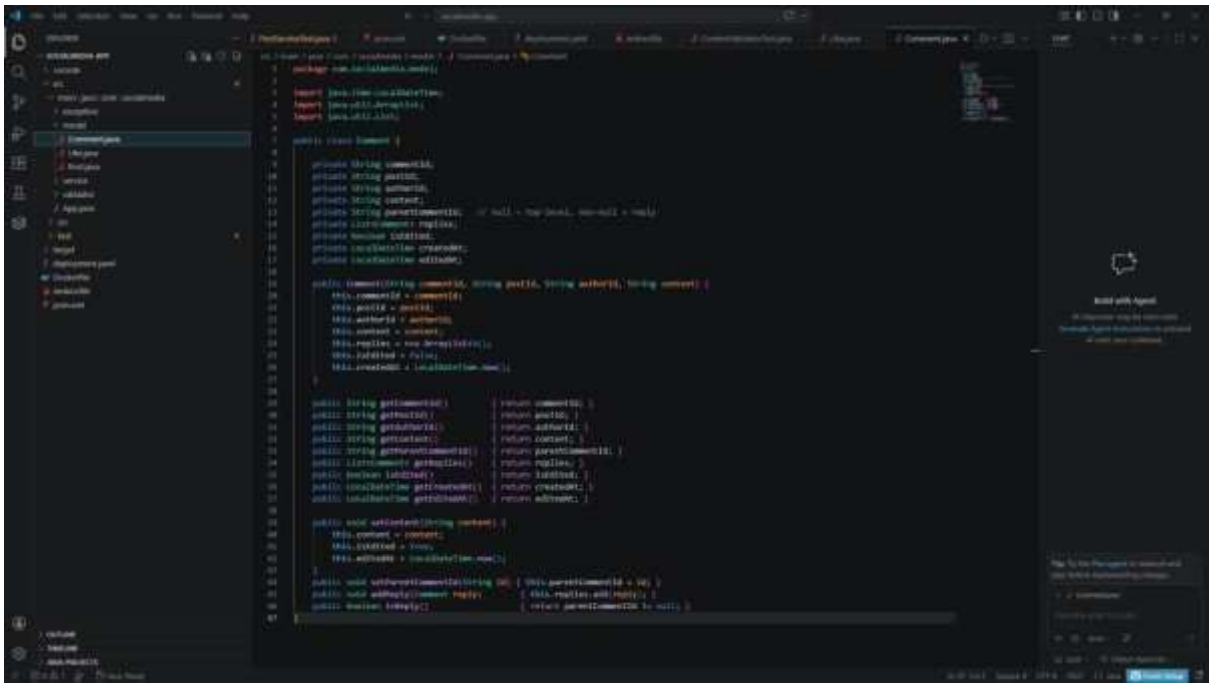
```
    @GetMapping("/status/{status}")     public ResponseEntity<List<Task>>
getTasksByStatus(@PathVariable TaskStatus status) {        List<Task> tasks =
taskService.getTasksByStatus(status);        return ResponseEntity.ok(tasks);
    }


    /**
* Update task progress.
     */
    @PutMapping("/{taskId}/progress")     public
ResponseEntity<Task> updateTaskProgress(
@PathVariable Long taskId,
            @RequestParam Integer hoursWorked) {
        Task updatedTask = taskService.updateTaskProgress(taskId, hoursWorked);
return ResponseEntity.ok(updatedTask);
    }


    /**
* Update task status.
     */
    @PutMapping("/{taskId}/status")     public
ResponseEntity<Task> updateTaskStatus(
            @PathVariable Long taskId,
            @RequestParam TaskStatus status) {
        Task updatedTask = taskService.updateTaskStatus(taskId, status);
return ResponseEntity.ok(updatedTask);
    }


    /**
* Get overdue tasks.
     */
    @GetMapping("/overdue")     public
ResponseEntity<List<Task>> getOverdueTasks() {
List<Task> tasks = taskService.getOverdueTasks();
return ResponseEntity.ok(tasks);
    }


    /**
```

```
* Delete a task.
    */
   @DeleteMapping("/{taskId}")     public ResponseEntity<Void>
deleteTask(@PathVariable Long taskId) {
taskService.deleteTask(taskId);        return
ResponseEntity.noContent().build();
   }


   /**
* Health check endpoint.
    */
   @GetMapping("/health")     public
ResponseEntity<String> health() {        return
ResponseEntity.ok("Task service is running");
   }
}
```

ProductTrackingApplication.java



Comment.java



PostNotFoundException.java package com.socialmedia.exception; public class PostNotFoundException extends RuntimeException { public PostNotFoundException(String postId) { super("Post not found: " + postId);

}

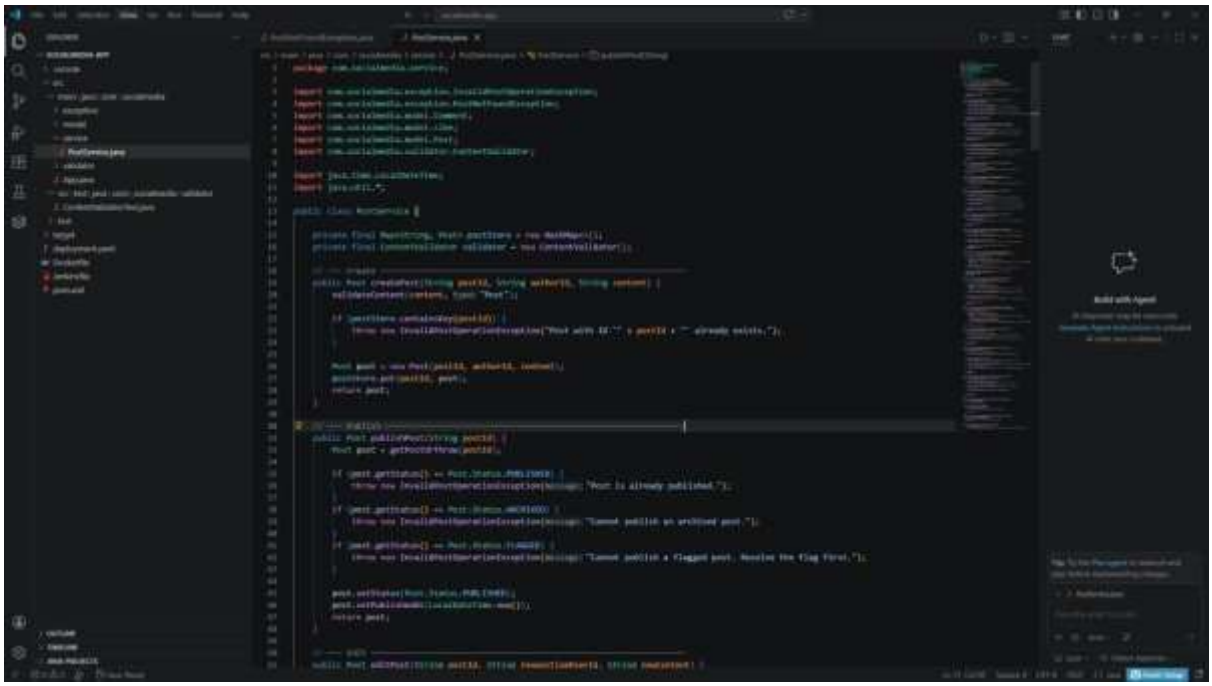
}

InvalidPostOperationException.java package com.socialmedia.exception; public class InvalidPostOperationException extends RuntimeException { public InvalidPostOperationException(String message) { super(message);

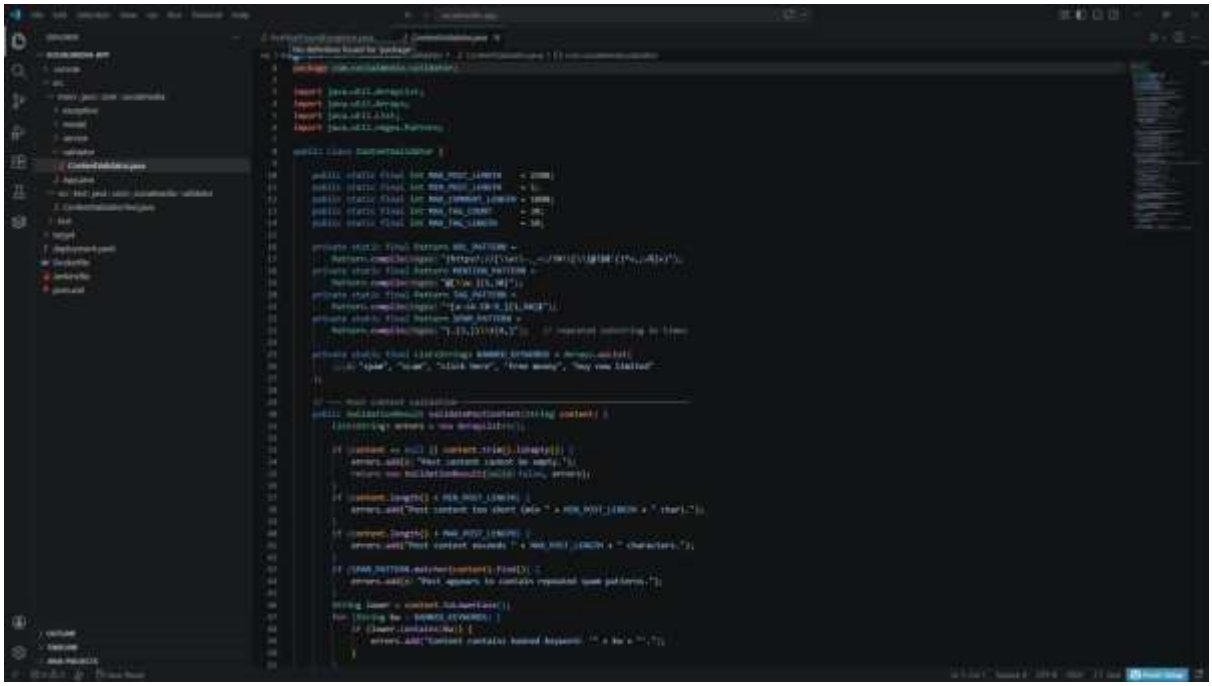
}

}

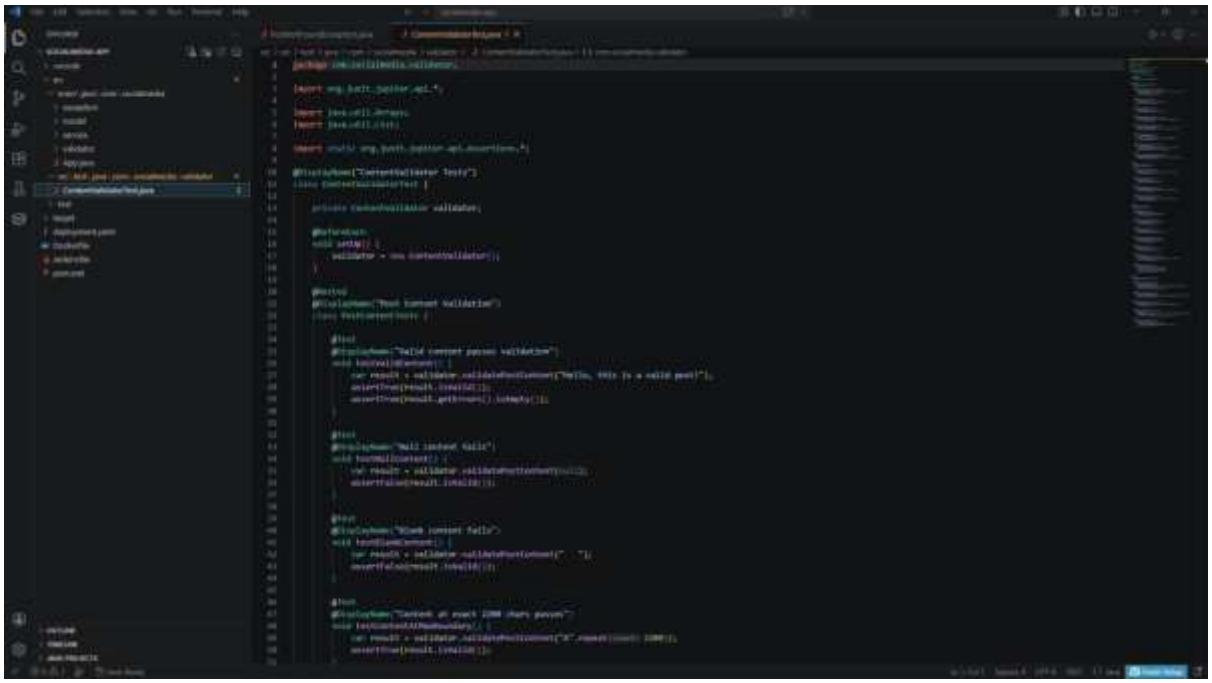
PostService.java



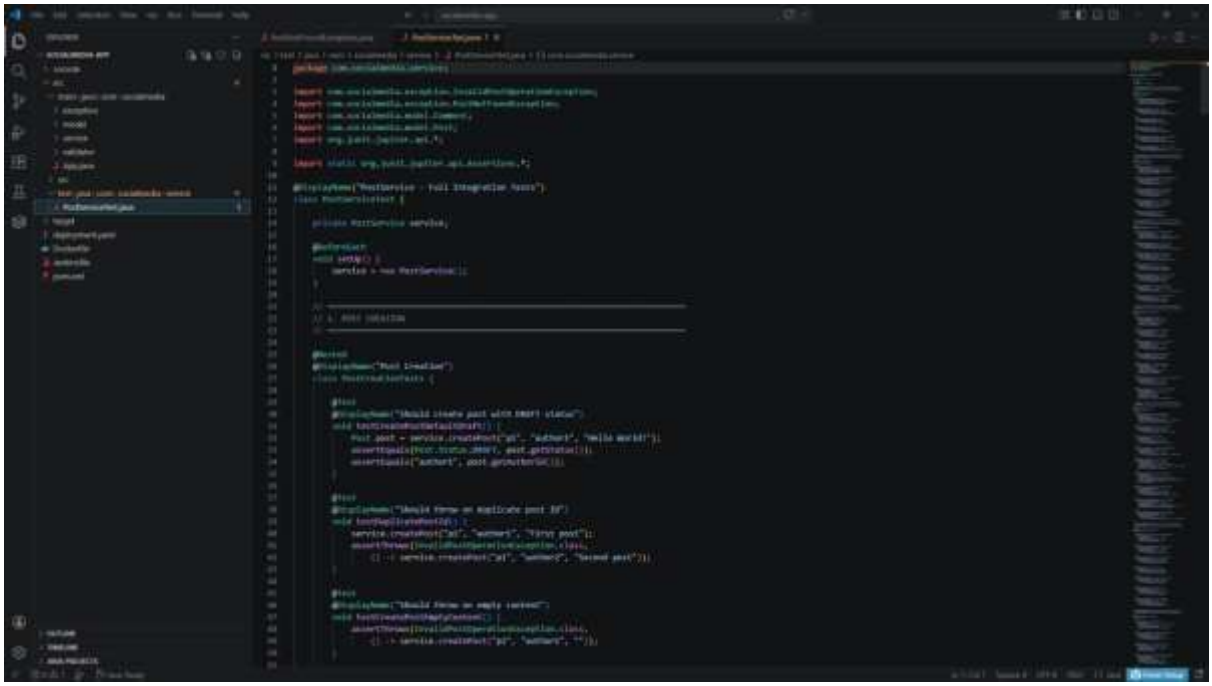
ContentValidator.java



ContentValidatorTest.java



PostServiceTest.java



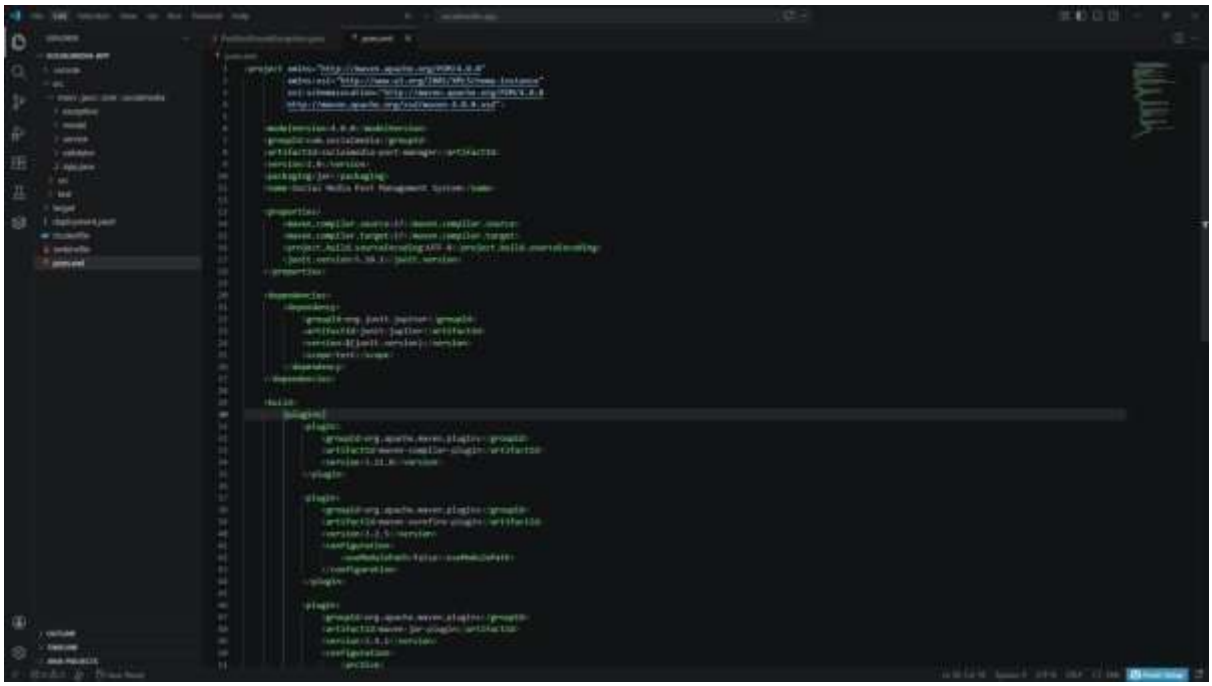
Deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: project-tracking-system
  namespace: default
  labels:
    app: project-tracking
    version: v1
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: project-tracking
  template:
    metadata:
      labels:
        app: project-tracking
        version: v1
    spec:
      containers:
      - name: project-tracking-system
        image: project-tracking-system:1.0.0
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
          name: http
          protocol: TCP
        env:
        - name: SPRING_PROFILES_ACTIVE
          value: "kubernetes"
        - name: SERVER_PORT
          value: "8080"
        - name: JAVA_OPTS
          value: "-Xms256m -Xmx512m"
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
```

Dockerfile

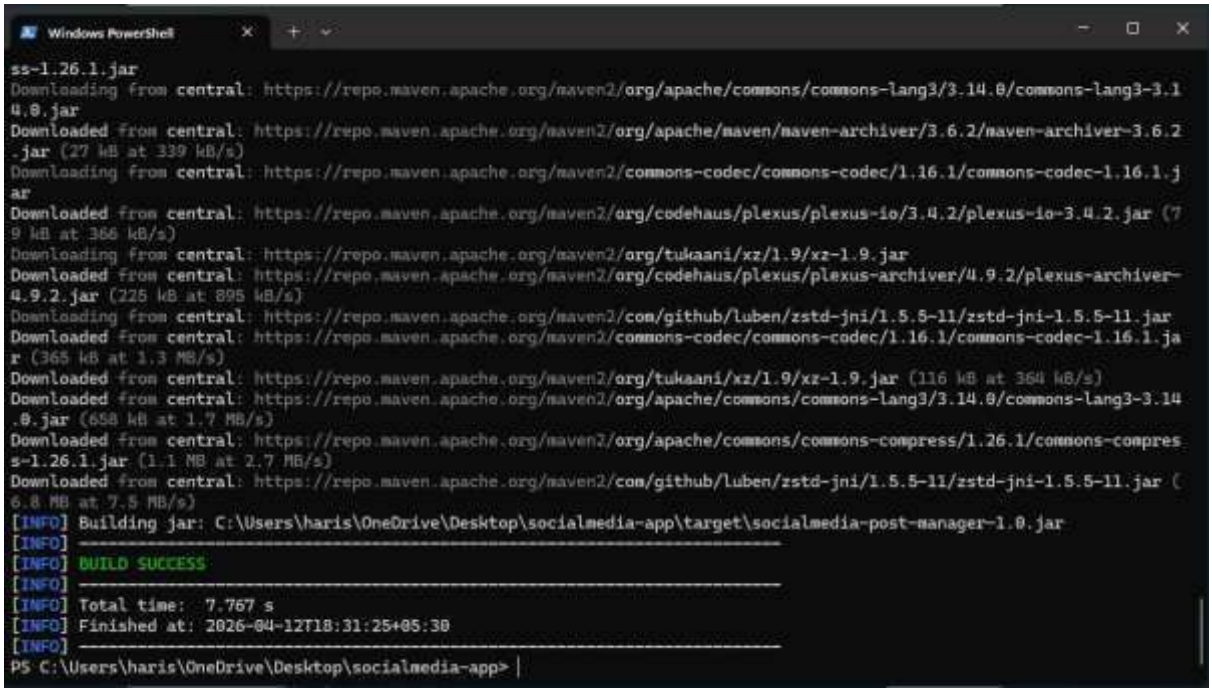
```
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Pom.xml

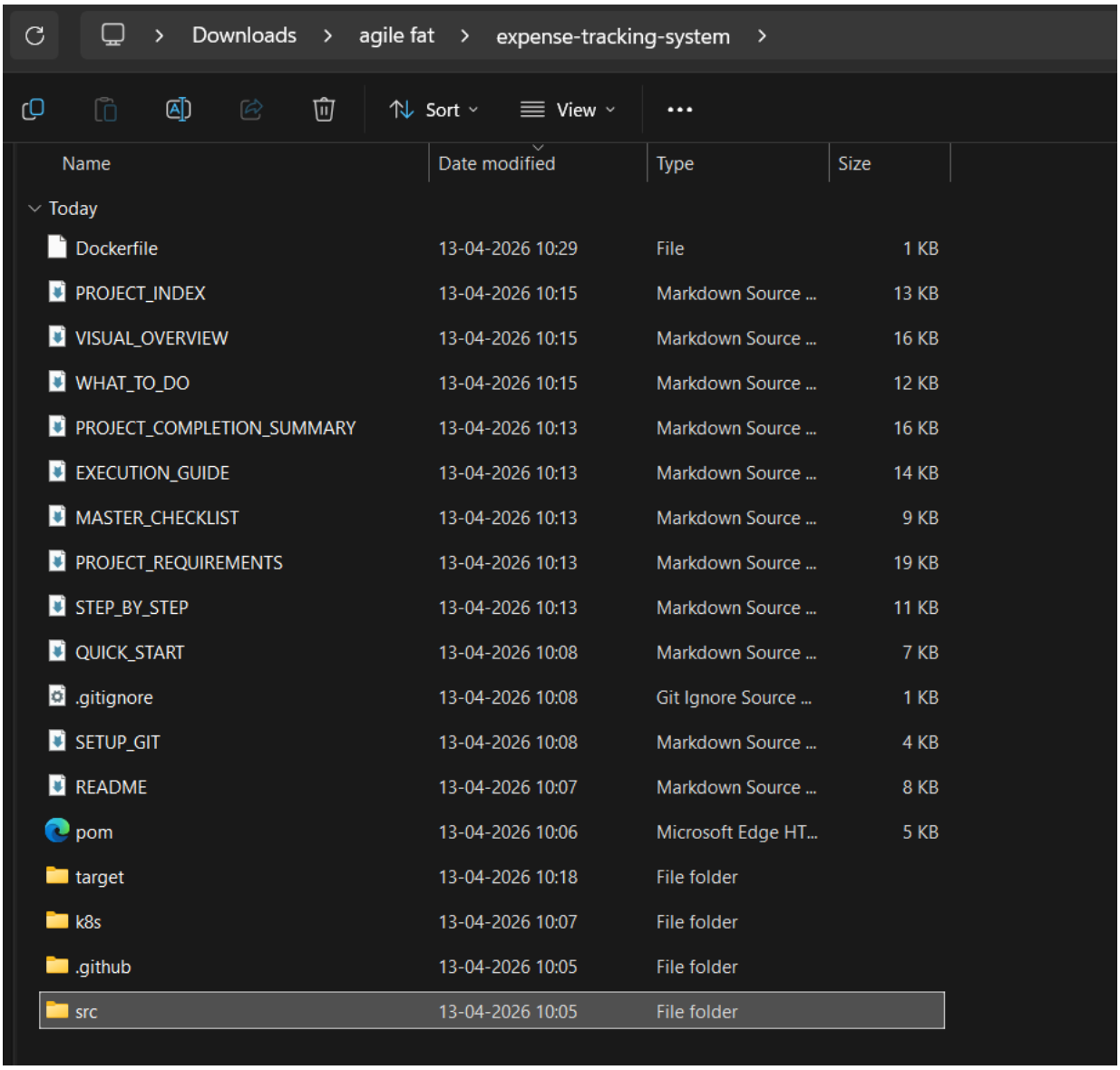


3. Build Locally using Maven (Test) mvn

clean test mvn clean package

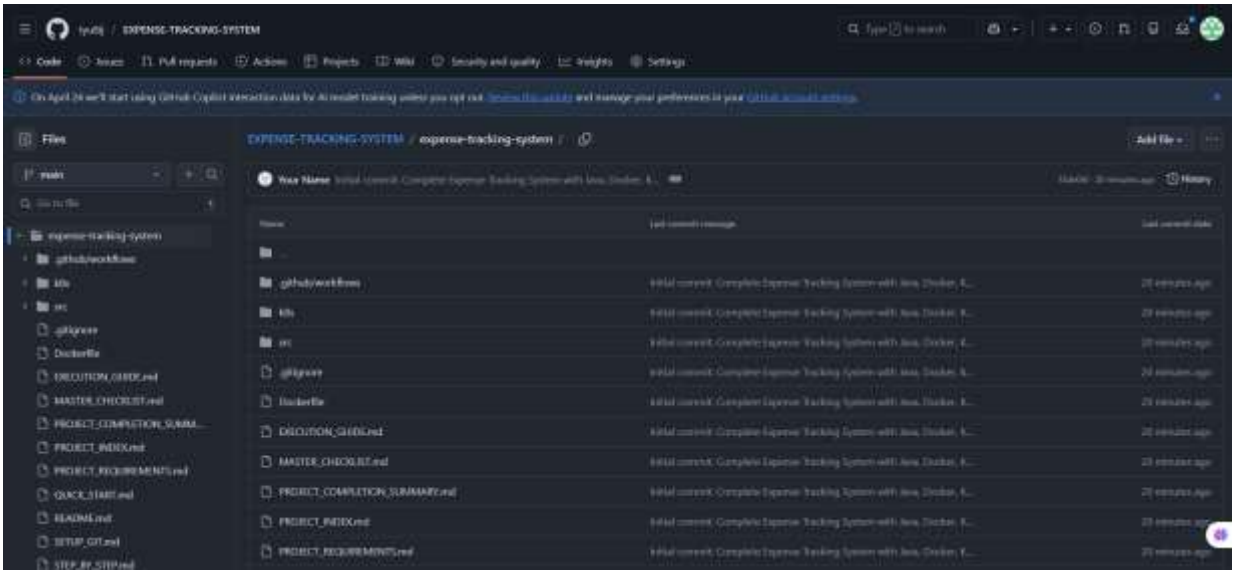


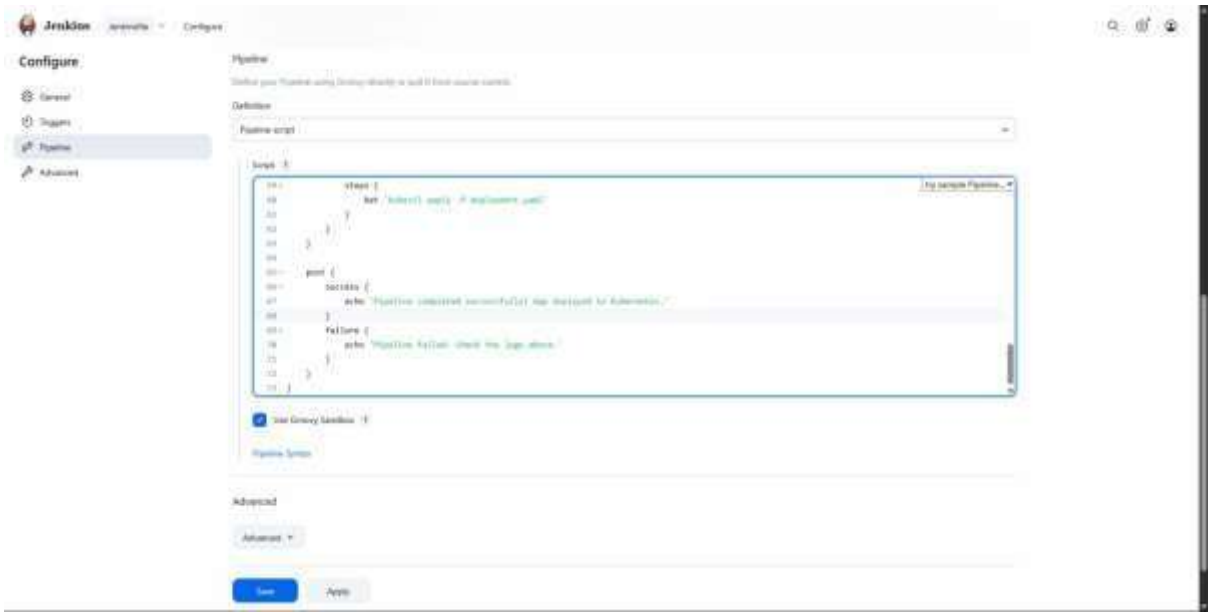
target/ test-java-app-1.0.jar



4. Push Code to GitHub

```
PS C:\Users\heman\Downloads\agile fat> git branch -M main
>> git push -u origin main
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 8 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (36/36), 49.79 KiB | 944.00 KiB/s, done.
Total 36 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/tyu8ij/EXPENSE-TRACKING-SYSTEM.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\heman\Downloads\agile fat> cd EXPENSE-TRACKING-SYSTEM/expense-tracking-system
>> mvn clean package
>> docker build -t expense-tracking-system:latest .
```





5. Create Jenkins Pipeline Job

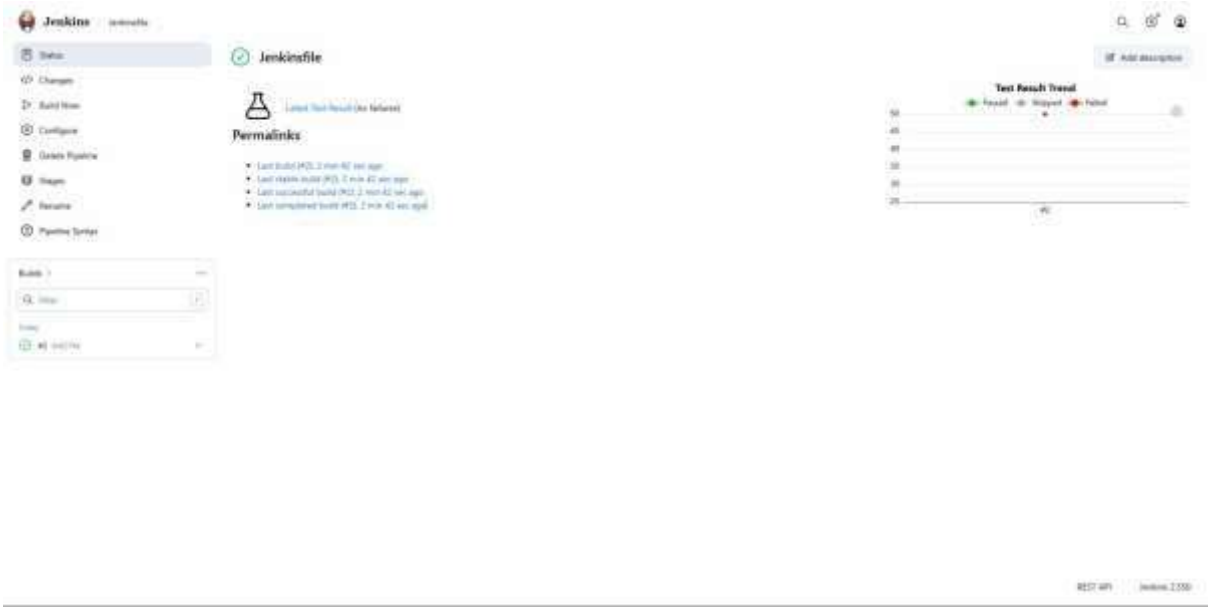
Pipeline Code

```
pipeline        {
agent        any
environment {
        DOCKER_IMAGE = "harish0805/socialmedia-post-manager:latest"
    }
```

```
    stages {         stage('Clone
Repository') {
            steps {                 git branch: 'main',                     url:
'https://github.com/codebyharish/socialmedia.git'
            }
        }
        stage('Build & Run JUnit Tests') {
            steps {                 bat
'mvn clean test'
            }
post {
always {                    junit 'target/surefire-
reports/*.xml'
                }
failure {                   echo 'Tests FAILED — aborting pipeline. No Docker image will
be built.'
                }
            }
        }
        stage('Package Application') {
            steps {                 bat 'mvn
package -DskipTests'
            }
        }
        stage('Build Docker Image') {
            steps {
                bat "docker build -t %DOCKER_IMAGE% ."
            }
        }
        stage('Push to DockerHub') {
```

```
        steps {                withCredentials([usernamePassword(
credentialsId: 'dockerhub-creds',                usernameVariable:
'DOCKER_USER',                passwordVariable: 'DOCKER_PASS'
            )]) {
bat """                    docker login -u %DOCKER_USER% -p
%DOCKER_PASS%
docker push %DOCKER_IMAGE%
                """
            }
        }
    }
    stage('Deploy to Kubernetes') {
        steps {
            bat 'kubectl apply -f deployment.yaml'
        }
    }
}
post {
success {            echo 'Pipeline completed successfully! App deployed to
Kubernetes.'
    }        failure {            echo 'Pipeline failed.
Check the logs above.'
    }
  }
}
```

Build the Pipeline

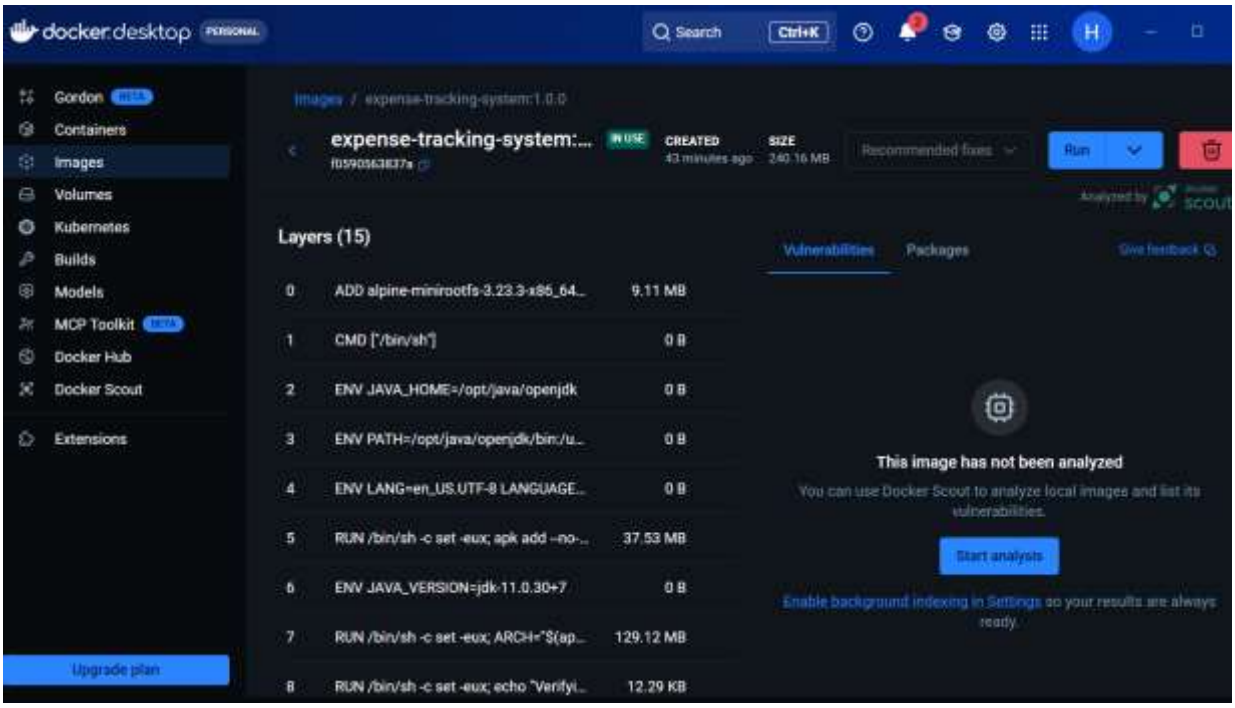

Jenkins
Jenkinsfile
Permalinks
Test Result Trend

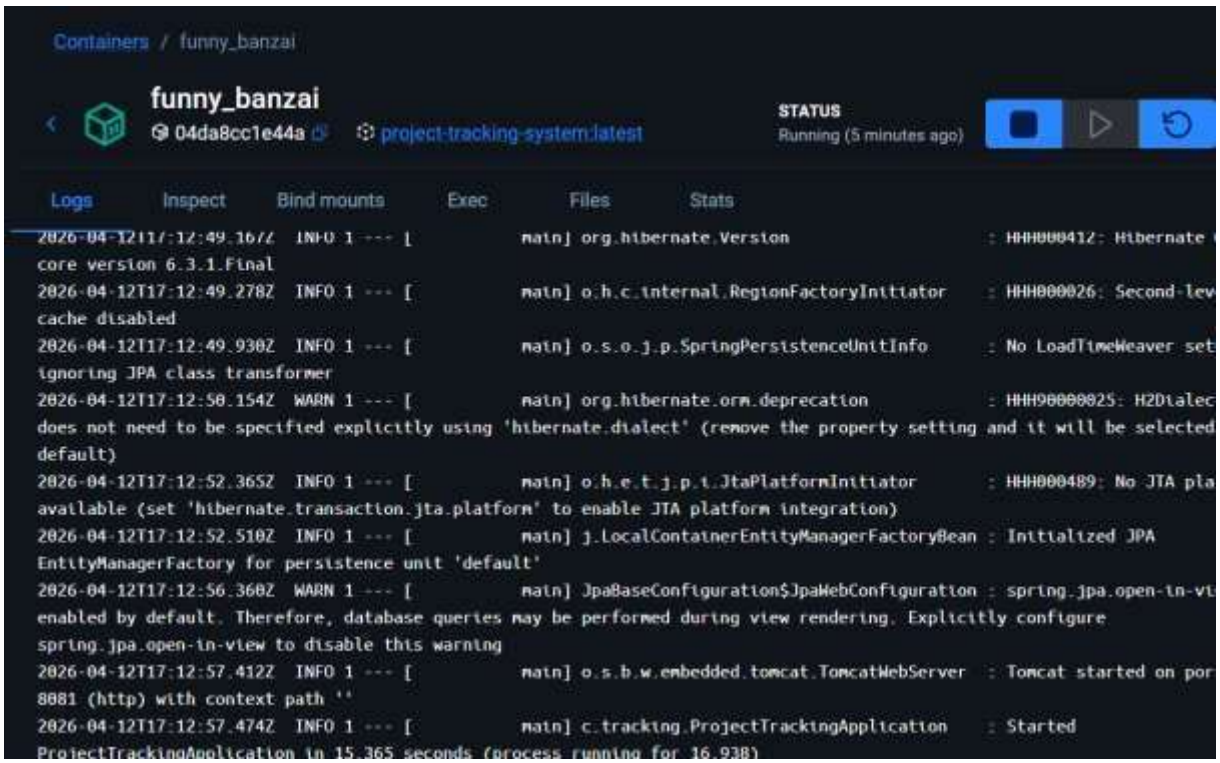
Jenkins
[INFO] Running com.socialmedia.service.PostServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.038 s -- in com.socialmedia.service.PostServiceTest
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
Post stage
Recording test results

Above console output is the success message

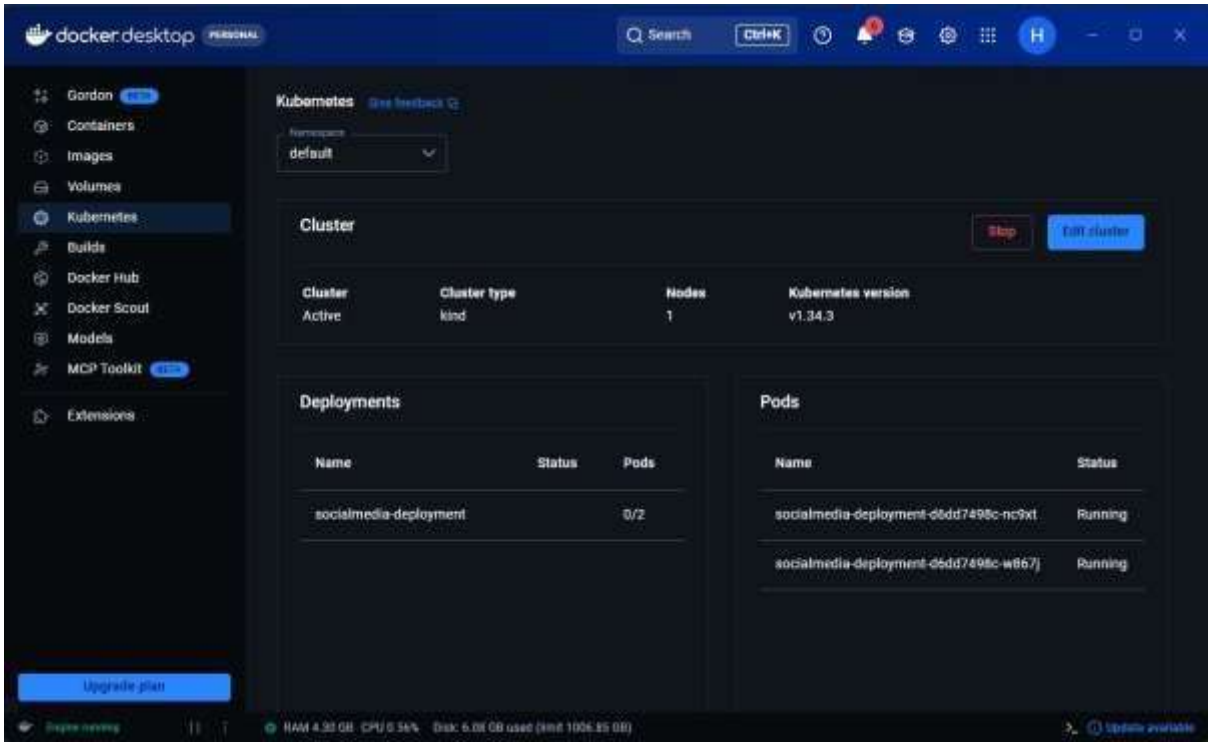
6. Verify Docker and Kubernetes Deployment

Docker Image




Containers / funny_banzai
funny_banzai
04da8cc1e44a
project-tracking-system:latest
STATUS
Running (5 minutes ago)
Logs
Inspect
Bind mounts
Exec
Files
Stats
2026-04-12T17:12:49.167Z INFO 1 --- [main] org.hibernate.Version : HHH000412: Hibernate
core version 6.3.1.Final
2026-04-12T17:12:49.278Z INFO 1 --- [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-lev
cache disabled
2026-04-12T17:12:49.930Z INFO 1 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver set
ignoring JPA class transformer
2026-04-12T17:12:50.154Z WARN 1 --- [main] org.hibernate.orm.deprecation : HHH90000025: H2Dialec
does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected
default)
2026-04-12T17:12:52.365Z INFO 1 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA pla
available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-04-12T17:12:52.510Z INFO 1 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA
EntityManagerFactory for persistence unit 'default'
2026-04-12T17:12:56.360Z WARN 1 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-vi
enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure
spring.jpa.open-in-view to disable this warning
2026-04-12T17:12:57.412Z INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on por
8081 (http) with context path ''
2026-04-12T17:12:57.474Z INFO 1 --- [main] c.tracking.ProjectTrackingApplication : Started
ProjectTrackingApplication in 15.365 seconds (process running for 16.938)

Kubernetes


docker desktop
Search
Ctrl+K
Gordon
Containers
Images
Volumes
Kubernetes
Builds
Docker Hub
Docker Scout
Models
MCP Toolkit
Extensions
Upgrade plan
Kubernetes
default
Cluster
Stop
Edit cluster
Cluster
Active
Cluster type
kind
Nodes
1
Kubernetes version
v1.34.3
Deployments
Name
Status
Pods
socialmedia-deployment
0/2
Pods
Name
Status
socialmedia-deployment-d6dd7498c-nc9xt
Running
socialmedia-deployment-d6dd7498c-w867j
Running

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\haris\OneDrive\Desktop\socialmedia-app> kubectl get svc
NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes            ClusterIP   10.96.0.1      <none>        443/TCP        17m
socialmedia-service   NodePort    10.96.96.219   <none>        80:30008/TCP   14m
PS C:\Users\haris\OneDrive\Desktop\socialmedia-app>
```

7.App.java OUTPUT

```
g back to 1 default profile: "default"
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-04-12T22:36:47.875+05:30  INFO 9988 --- [          main] o.s.b.t.m.w.SpringBootMockServletContext : Initializing Spring TestDispa
tcherServlet ''
2026-04-12T22:36:47.875+05:30  INFO 9988 --- [          main] o.s.t.web.servlet.TestDispatcherServlet  : Initializing Servlet ''
2026-04-12T22:36:47.878+05:30  INFO 9988 --- [          main] o.s.t.web.servlet.TestDispatcherServlet  : Completed initialization in 3
 ms
2026-04-12T22:36:47.919+05:30  INFO 9988 --- [          main] com.tracking.TaskControllerTest          : Started TaskControllerTest in
 9.544 seconds (process running for 11.884)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 11.28 s -- in com.tracking.TaskControllerTest
[INFO] Running com.tracking.TaskServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.697 s -- in com.tracking.TaskServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ project-tracking-system ---
[INFO] Building jar: C:\Users\heman\Downloads\Project-Tracking-System\target\project-tracking-system-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.2.0:repackage (repackage) @ project-tracking-system ---
[INFO] Replacing main artifact C:\Users\heman\Downloads\Project-Tracking-System\target\project-tracking-system-0.0.1-SNAPSHOT.jar with
repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\heman\Downloads\Project-Tracking-System\target\project-tracking-system-0.0.1-
SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  22.659 s
[INFO] Finished at: 2026-04-12T22:36:51+05:30
[INFO] ------------------------------------------------------------------------
[+] Building 82.8s (8/8) FINISHED                                                                                  docker:desktop-linux
```

```
------------------------------------------------------------
  -> post_001 shares: 2
  -> post_002 shares: 1

------------------------------------------------------------
  7. PINNING POST
------------------------------------------------------------
  -> post_001 pinned: true

------------------------------------------------------------
  8. EDITING POST
------------------------------------------------------------
  -> post_003 updated content: Agile development means shipping fast, testing faster, and learning always!

------------------------------------------------------------
  9. FLAGGING POST
------------------------------------------------------------
Exception in thread "main" com.socialmedia.exception.InvalidPostOperationException: Post is already published.
        at com.socialmedia.service.PostService.publishPost(PostService.java:36)
        at com.socialmedia.App.main(App.java:98)
PS C:\Users\haris\OneDrive\Desktop\socialmedia-app>
```